

-1

بخش اول:

در ابتدا چون هنوز چیزی از اکشن های بازی مشخص نیست عامل نمیداند که چه کاری دقیقا خوب است به همین خاطر در اول کار میانگین ریوارد کم و حتی منفی است، اما با گذر زمان و با کاهش ریت **exploration** ، مدل یاد میگیرد کار خوب چیست (با استفاده از الگوی الگوریتم **Q-learning**) و از یک جایی به بعد که به ریوارد بالایی رسید با واریانس نسبتا کم بالا پایین میشود چون ریت اسپیلن دیگه خیلی میاد پایی و ایجنت پایدار تر عمل میکند (اول واریانس بیشتر هرچی بیشتر میگذره و بیشتر میریم جلو واریانس کمتر میشه)

بخش دوم:

در ابتدا با درصد موفقیت 3 درصد شروع میکنیم بعد میشه 8 درصد و با گذر زمان کم کم بیشتر میشود، در اینجا هم از یکجایی به بعد بخاطر کم شدن نرخ **exploration** واریانس تغییر کم میشود و با زیاد شدن اوریج موفقیت تقریبا نزدیک 98-99 میماند و با واریانس نسبتا پایین حول این میانگین بالا پایین میشود. پس با افزایش تعداد اپیزود ها (هرچی جلوتر میریم) اسپیلون پایین میاد و مدل کارهای موفق تضمینی را انجام میدهد و بعد از یکجایی اکثر اپیزود ها ریت موفقیت بالایی میگیرند.

بخش سوم:

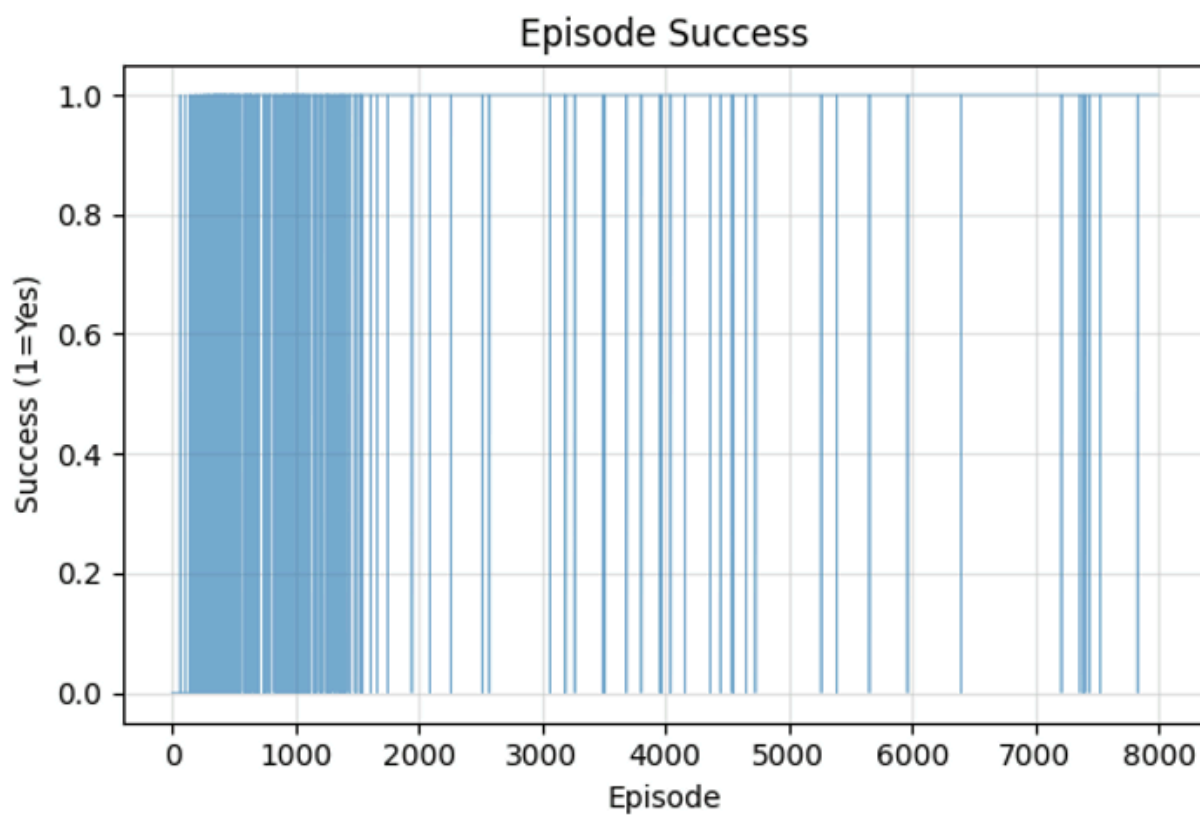
این مقدار نیز در ابتدا بسیار زیاد است (مثلا در شروع 197.1 است) اما با گذر زمان رفته رفته اوریج استپ کم میشه و بخاطر همان کم شدن ریت گاما و بالطبع کاهش **exploration** ، کارهای بهتر سریع تر انتخاب میشوند و به همین خاطر ب استپ های کمتری نیاز است، اما این روند نیز نزولی اکیدی نیست و بعد از یک جایی حول یک محوری با واریانس نسبتا پایینی جابجا میشود و در آخر تقریبا به میانگین 15 تا رسیده و بین بازه 12 تا 17 جابجا میشود.

-2
نمودار اول:



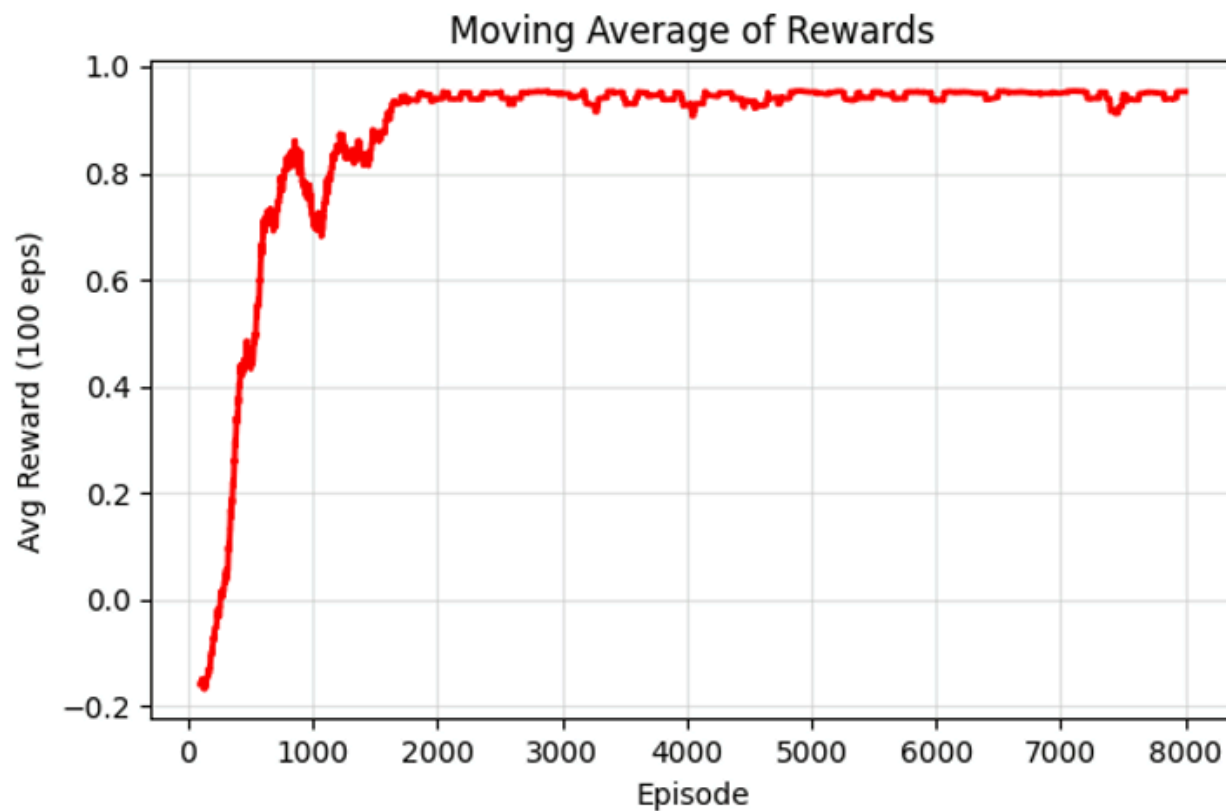
همانطور که انتظار میرفت با جلو رفتن تعداد اپزیدود ها میانگین **total reward** رفته رفته بالا میرود و واریانس آن نیز کم میشود و تقریباً حول مقدار 0.9 تا 1 جابجا میشود، همچنین هرازگاهی دیده میشود که ریوارد های پایینی کسب میکند که این هم رفته رفته ریتش کم میشه که دلیلش کم شدن گاما و بالطبع کم شدن **exploration** است.

نمودار دوم:



این هم تقریباً مانند نمودار قبلیست و با گذر زمان تقریباً همیشه YES است با success rate نسبتاً بالا که خب این هم با گذر زمان ثابت میشود و باز هم اون یک ذره مقادیر پایینی که داره پیدا میکنه بخاطر همون ریت گاما است که میبینیم این هم ریتش با گذر زمان کمتر میشه و success بیشتری میگیریم.

نمودار سوم:



اولش میانگین ریواردمون نسبتاً پایینه و بعدش یهویی افزایش پیدا میکنه و بعد از یه مدتی مقدارش ثابت میمونه و نزدیک به ماکزیمم، میبینیم که اخراش خیلی نویز نداریم که این رسیدن به حالت پایدار نشون دهنده همگرا بودن الگوریتم Q-learning و بعد از یه مدتی به پالیسی بهینه میریسه و بعدش باز با یه واریانس خیلی کمی بالا پایین میشود.

نمودار چهارم:



اول اولش نرخ success تقریبا صفره و رفته رفته سرعتش زیاد میشه و اخراش پایدار میشه و نزدیک به صد میشه این نمودار نشون میده بعد از اینکه یادگیری انجام میشه مدل توی اپیزود های اخر مخصوصا موفق عمل میکنه و همگرایی هم میبینیم که نشون میده پالیسی ما پایدار و قابل استفاده است اون اخراشم که نوسان تغییر کم میشه و اروم بالا پایین میشه بخاطر اپسیلون گریدی بودن الگوریتم و رفتار غیرقطعی و exploration است.

سوال ۳.

(برای پاسخ به سوالات این بخش تابع `select_action` کمک کننده خواهد بود.)

مقدار `epsilon` چه تغییری در حین آموزش کرده است؟

در ابتدای کار مقدار `epsilon` بالا است و رفته رفته کم میشود و از بجایی به بعد روی یک صدم ثابت میشود. مثلاً توی مدل ما از اپیزود 500 به بعد روی 0.01 ثابت شده است

مقدار آن چه تاثیری در فرایند آموزش دارد؟

از اونجایی که اپسیلون تعیین میکنه برامون که احتمال انتخاب اکشن تصادفی چقدره، اون اول کار و زمان بالا بودن اپسیلون طبیعتاً عامل `exploration` بیشتری دارد و فضای حالتمون بهتر کشف میشه ولی با گذر زمان و کم شدن اپسیلون عامل بیشتر `exploit` میکنه و رفتار عامل پایدار تر و نزدیک تر و همگرا به سیاستی که پیدا کرده که بهینه هم هست میرسه پس نقش و تاثیر این مقدار ایجاد تعادل بین `exploration` و `exploitation` است

دلیل استفاده از `epsilon_decay_rate` در فرایند آموزش چیست؟

هدفش در اول کار اینه که عاملمون فرصت برای یادگیری محیطمون داشته باشه و بعد از گذشت زمان و بهتر شدن مقادیر `Q` مون نیازمون به `exploration` کاهش پیدا کنه و فرایند لرنمون کم کم همگرا بشه به سیاست درست

اگه اپسیلون ثابت بمونه تغییر نکنه و مثلاً بزرگ بشه لرن همیشه رندومه و سیاست بهتر پیدا نمیشه، اگه اپسیلون کوچیک باشه عامل ممکنه تو یه سیاست غیر بهینه گیر کنه پس نیاز داریم که اپسیلون تدریجی کاهش پیدا کنه که بازم برای تعادله

چرا در تابع `test` (هنگام ارزیابی عامل پس از آموزش) مقدار `epsilon` صفر لحاظ شده است؟

زیرا تست بعد از لرن داره اتفاق میفته و در طول لرن سیاست بهینه پیدا شده و نیاز نداریم تغییر بدیم همونطور که گفته شد در طول ارائه طبق الگوریتم `Q-learning`

اپسیلون کم باشه ما سر سیاست میونیم و اصلاً اکشن جدید انتخاب نمیکنیم و عملاً `exploration` نداریم. دلایلش هم اینه که خب سیاست بهینه رو داریم نیاز نیست اصلاً بریم چیزی پیدا کنیم و همچی بر اساس `Q-table` کاملاً `greedy` انتخاب میشه

سوال ۴.

اگر در توابع train و test به جای

`env.reset()`

قرار دهیم:

`env.reset(seed=42)`

باعث تکرار پذیری محیط در هر اپیزود می شود.

این کار چه تغییری در آموزش عامل ایجاد می کند؟

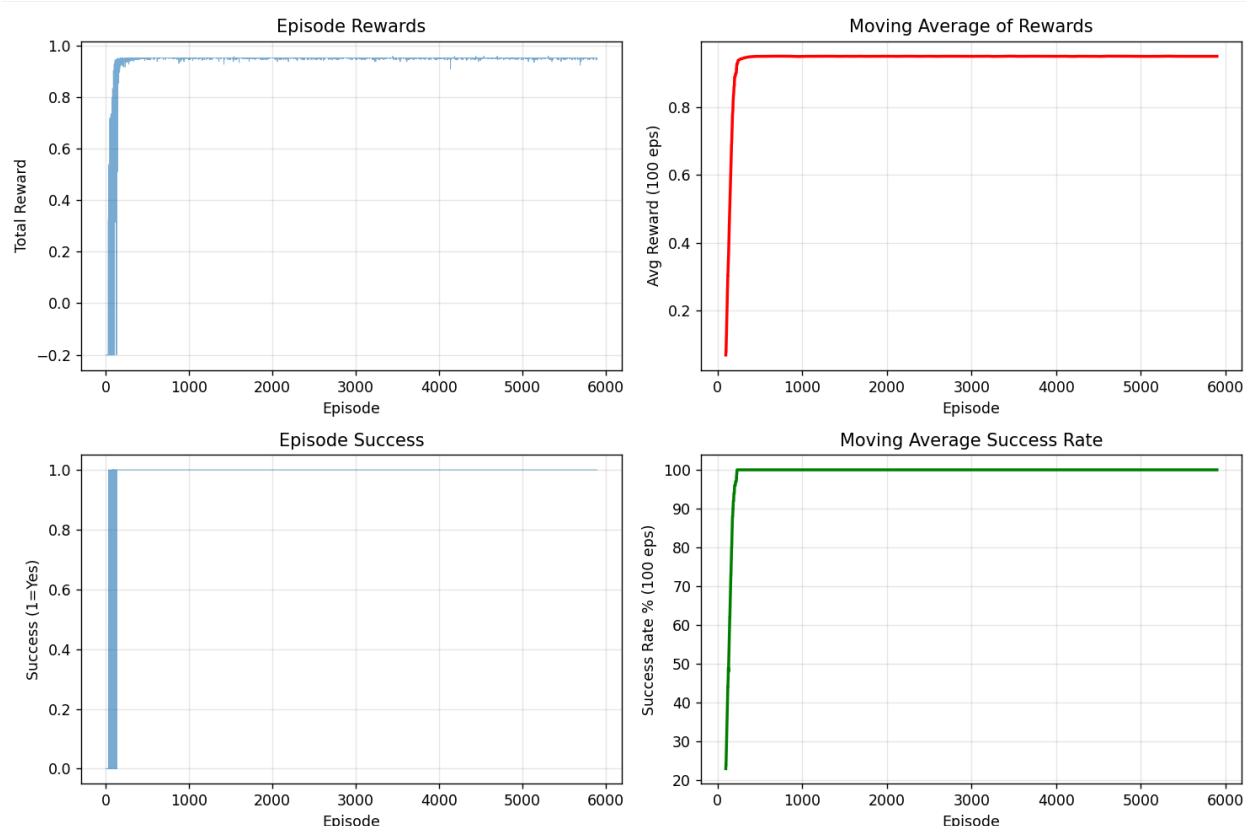
۴ نمودار خروجی پس از آموزش را تحلیل کنید.

ما اینکارو کردیم و بعد الان خروجی شد این:

```
Action Space: 7
Start Training.....
Episode 100, Epsilon: 0.366, Avg Reward: 0.068, Success: 23.0%, Avg Steps: 171.0, States: 298
Episode 200, Epsilon: 0.134, Avg Reward: 0.862, Success: 94.0%, Avg Steps: 33.8, States: 311
Episode 300, Epsilon: 0.049, Avg Reward: 0.944, Success: 100.0%, Avg Steps: 15.9, States: 311
Episode 400, Epsilon: 0.018, Avg Reward: 0.949, Success: 100.0%, Avg Steps: 14.5, States: 311
Episode 500, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 600, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 700, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 800, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.0, States: 311
Episode 900, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 1000, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.3, States: 311
Episode 1100, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 1200, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 1300, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 1400, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 1500, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 1600, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 1700, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 1800, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 1900, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 2000, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 2100, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 2200, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 2300, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 2400, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 2500, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 2600, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 2700, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 2800, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 2900, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 3000, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 3100, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 3200, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 3300, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.3, States: 311
Episode 3400, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 3500, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 3600, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 3700, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 3800, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 3900, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 4000, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 311
Episode 4100, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 311
Episode 4200, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 312
Episode 4300, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 312
Episode 4400, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 312
Episode 4500, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 312
Episode 4600, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 312
Episode 4700, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 312
Episode 4800, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 312
Episode 4900, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 312
Episode 5000, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 312
Episode 5100, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 312
Episode 5200, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 312
Episode 5300, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 312
Episode 5400, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 312
Episode 5500, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 312
Episode 5600, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 312
Episode 5700, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 313
Episode 5800, Epsilon: 0.010, Avg Reward: 0.951, Success: 100.0%, Avg Steps: 14.1, States: 313
Episode 5900, Epsilon: 0.010, Avg Reward: 0.950, Success: 100.0%, Avg Steps: 14.2, States: 313
```

چون جای کلید و در الان ثابت به همین خاطر دیگه مدل هی روی محیط های متفاوت نیاز نیست چیزی یاد بگیره و از اونجایی که محیط ثابت میمونه و چیشش یکسانه و شرایط اولیه تغییر نمیکنه عاملمون فقط یک مسیر خاصی از محیط رو یاد میگیره و تنوع هامون کاملاً کاهش پیدا میکنه و از یجایی به بعد باعث حفظ کردن روش حل میشه نه یادگیری روش حل و از exploration و explotation به memrization میرسیم و این خوب نیست و روی محیط های دیگه خیلی خوب جواب نخواهد داد به اندازه قبلی الان سیاستمون ثابت میشه و به همین خاطر exploration واقعی خیلی کاهش پیدا میکنه و تأثیرش روی مدل ترین اینه که توی ترین خیلی شاهکار عمل میکنه ولی توی تست اگه محیط متفاوت باشه ممکنه عملکرد ضعیفی داشته باشه

تأثیرش روی تست هم اینه که اگه داخل تست هم از $seed=42$ استفاده بشه همون محیطی میشه که ترین دیده و ارزیابیمون دیگه اصلاً واقعی نخواهد بود و عملکرد زیادی خوب بنظر میاد ولی هدف تست تست کردن عمومی عامل بود نه تست کردنش روی محیط خاصی



خب $total\ reward$ خیلی سریع افزایش پیدا میکنه و بعد دیگه اصلاً حتی واریانس هم خیلی خیلی کم میشه و اصلاً $explore$ نمینه و از یجایی به بعد عملاً حفظ میکنه کار خوبی رو، $success\ rate$ خیلی سریع زیاد میشه و دیگه نوسان نداره $success$ اش در اکثر مواقع **Yes** و یک است و به طور خلاصه همه نمودار ها خیلی سریع پایدار میشن و همونطور که گفتیم مدل عملاً محیط رو حفظ میکنه و یاد نمیگیره

پس به صورت خلاصه استفاده از $seed=42$ باعث تکرار پذیری محیط میشه و تنوع تجربه ها کاهش پیدا میکنه و یادگیری محدود میشه و عامل بجای یادگیری عمومی یک مسیر خاصی رو حفظ میکنه، برای ارزیابی میتونیم $seed$ ندیم به تست که ببینیم چقدر احمقانه عمل میکنه نسبت به ترین قبلی بدون $seed$

1. مقدار $\text{seed}=42$ تضمین می‌کند که تمام randomness در طول کل اپیزود کاملاً deterministic باشد.

این جمله غلطه، این seed گذاشتن درسته که میاد شرایط اولیه محیط رو تکرار پذیر می‌کنه اما تمام عوامل randomness بودن رو حذف نمی‌کنه درحالی که جمله گفته "تمام". مثلاً ما سیاست انتخاب اکشن تصادفی رو داخل `np.random` داریم هنوز و در برخی محیط های خاص در حین اپیزود ممکنه randomness داشته باشیم پس اپیزود هنوز هم میتونه رفتار غیرقطعی داشته باشه

2. در سیستم های RL همواره باید از مقدار $\text{seed}=42$ استفاده کرد.

اینم غلطه و لزومی نداره، این برای دیباگ کردن و چیزهایی از این قبیل در مسائل RL استفاده میشه و درواقع در ترین اگه seed تعیین کنیم دیگه اصلاً ترینمون تقویتی نیست و یادگیری نداریم و فقط حفظ میکنیم. درواقع از این کار برای دیباگ و مقایسه الگوریتم‌ها و اینجور کارا مناسبه.

نداشتن تست seed حالت

Testing learned policy.....

Test Episode 1

Steps: 100, Reward: 0.000, Success: No

Test Episode 2

Steps: 100, Reward: 0.000, Success: No

Test Episode 3

Steps: 100, Reward: 0.000, Success: No

Test Episode 4

Steps: 100, Reward: 0.000, Success: No

Test Episode 5

Steps: 100, Reward: 0.000, Success: No

Test Summary: 0/5 successful (0.0%), Average Steps: 100.0

میبینیم که خیلی بد عمل میکنه توی محیط های دیگر